

- Gill, P.E., Murray, W., and Wright, M.H. 1991, *Numerical Linear Algebra and Optimization*, vol. 1 (Redwood City, CA: Addison-Wesley).
- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), Chapter 4.
- Ueberhuber, C.W. 1997, *Numerical Computation: Methods, Software, and Analysis*, 2 vols. (Berlin: Springer), Chapter 13.
- Coleman, T.F., and Van Loan, C. 1988, *Handbook for Matrix Computations* (Philadelphia: S.I.A.M.).
- Forsythe, G.E., and Moler, C.B. 1967, *Computer Solution of Linear Algebraic Systems* (Englewood Cliffs, NJ: Prentice-Hall).
- Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).
- Westlake, J.R. 1968, *A Handbook of Numerical Matrix Inversion and Solution of Linear Equations* (New York: Wiley).
- Johnson, L.W., and Riess, R.D. 1982, *Numerical Analysis*, 2nd ed. (Reading, MA: Addison-Wesley), Chapter 2.
- Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), Chapter 9.

2.1 Gauss-Jordan Elimination

Gauss-Jordan elimination is probably the way you learned to solve linear equations in high school. (You may have learned it as “Gaussian elimination,” but, strictly speaking, that term refers to the somewhat different technique discussed in §2.2.) The basic idea is to add or subtract linear combinations of the given equations until each equation contains only one of the unknowns, thus giving an immediate solution. You might also have learned to use the same technique for calculating the inverse of a matrix.

For solving sets of linear equations, Gauss-Jordan elimination produces *both* the solution of the equations for one or more right-hand side vectors $\mathbf{b}$, and also the matrix inverse $\mathbf{A}^{-1}$. However, its principal deficiencies are (i) that it requires all the right-hand sides to be stored and manipulated at the same time, and (ii) that when the inverse matrix is *not* desired, Gauss-Jordan is three times slower than the best alternative technique for solving a single linear set (§2.3). The method’s principal strength is that it is as stable as any other direct method, perhaps even a bit more stable when full pivoting is used (see §2.1.2).

For inverting a matrix, Gauss-Jordan elimination is about as efficient as any other direct method. We know of no reason not to use it in this application if you are sure that the matrix inverse is what you really want.

You might wonder about deficiency (i) above: If we are getting the matrix inverse anyway, can’t we later let it multiply a new right-hand side to get an additional solution? This does work, but it gives an answer that is very susceptible to roundoff error and not nearly as good as if the new vector had been included with the set of right-hand side vectors in the first instance.

Thus, Gauss-Jordan elimination should not be your method of first choice for solving linear equations. The decomposition methods in §2.3 are better. Why do we discuss Gauss-Jordan at all? Because it is straightforward, solid as a rock, and a good place for us to introduce the important concept of *pivoting* which will also

be important for the methods described later. The actual sequence of operations performed in Gauss-Jordan elimination is very closely related to that performed by the routines in the next two sections.

2.1.1 Elimination on Column-Augmented Matrices

For clarity, and to avoid writing endless ellipses ($\cdots$) we will write out equations only for the case of four equations and four unknowns, and with three different right-hand side vectors that are known in advance. You can write bigger matrices and extend the equations to the case of $N \times N$ matrices, with M sets of right-hand side vectors, in completely analogous fashion. The routine implemented below in §2.1.2 is, of course, general.

Consider the linear matrix equation

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix} \cdot \left[\begin{pmatrix} x_{00} \\ x_{10} \\ x_{20} \\ x_{30} \end{pmatrix} \sqcup \begin{pmatrix} x_{01} \\ x_{11} \\ x_{21} \\ x_{31} \end{pmatrix} \sqcup \begin{pmatrix} x_{02} \\ x_{12} \\ x_{22} \\ x_{32} \end{pmatrix} \sqcup \begin{pmatrix} y_{00} & y_{01} & y_{02} & y_{03} \\ y_{10} & y_{11} & y_{12} & y_{13} \\ y_{20} & y_{21} & y_{22} & y_{23} \\ y_{30} & y_{31} & y_{32} & y_{33} \end{pmatrix} \right] \\ = \left[\begin{pmatrix} b_{00} \\ b_{10} \\ b_{20} \\ b_{30} \end{pmatrix} \sqcup \begin{pmatrix} b_{01} \\ b_{11} \\ b_{21} \\ b_{31} \end{pmatrix} \sqcup \begin{pmatrix} b_{02} \\ b_{12} \\ b_{22} \\ b_{32} \end{pmatrix} \sqcup \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \right] \quad (2.1.1)$$

Here the raised dot ($\cdot$) signifies matrix multiplication, while the operator $\sqcup$ just signifies column augmentation, that is, removing the abutting parentheses and making a wider matrix out of the operands of the $\sqcup$ operator.

It should not take you long to write out equation (2.1.1) and to see that it simply states that x_{ij} is the i th component ($i = 0, 1, 2, 3$) of the vector solution of the j th right-hand side ($j = 0, 1, 2$), the one whose coefficients are b_{ij} , $i = 0, 1, 2, 3$; and that the matrix of unknown coefficients y_{ij} is the inverse matrix of a_{ij} . In other words, the matrix solution of

$$[\mathbf{A}] \cdot [\mathbf{x}_0 \sqcup \mathbf{x}_1 \sqcup \mathbf{x}_2 \sqcup \mathbf{Y}] = [\mathbf{b}_0 \sqcup \mathbf{b}_1 \sqcup \mathbf{b}_2 \sqcup \mathbf{1}] \quad (2.1.2)$$

where $\mathbf{A}$ and $\mathbf{Y}$ are square matrices, the $\mathbf{b}_i$'s and $\mathbf{x}_i$'s are column vectors, and $\mathbf{1}$ is the identity matrix, simultaneously solves the linear sets

$$\mathbf{A} \cdot \mathbf{x}_0 = \mathbf{b}_0 \quad \mathbf{A} \cdot \mathbf{x}_1 = \mathbf{b}_1 \quad \mathbf{A} \cdot \mathbf{x}_2 = \mathbf{b}_2 \quad (2.1.3)$$

and

$$\mathbf{A} \cdot \mathbf{Y} = \mathbf{1} \quad (2.1.4)$$

Now it is also elementary to verify the following facts about (2.1.1):

- Interchanging any two *rows* of $\mathbf{A}$ and the corresponding *rows* of the $\mathbf{b}$'s and of $\mathbf{1}$ does not change (or scramble in any way) the solution $\mathbf{x}$'s and $\mathbf{Y}$. Rather, it just corresponds to writing the same set of linear equations in a different order.
- Likewise, the solution set is unchanged and in no way scrambled if we replace any row in $\mathbf{A}$ by a linear combination of itself and any other row, as long as we do the same linear combination of the rows of the $\mathbf{b}$'s and $\mathbf{1}$ (which then is no longer the identity matrix, of course).

- Interchanging any two *columns* of $\mathbf{A}$ gives the same solution set only if we simultaneously interchange corresponding *rows* of the $\mathbf{x}$'s and of $\mathbf{Y}$. In other words, this interchange scrambles the order of the rows in the solution. If we do this, we will need to unscramble the solution by restoring the rows to their original order.

Gauss-Jordan elimination uses one or more of the above operations to reduce the matrix $\mathbf{A}$ to the identity matrix. When this is accomplished, the right-hand side becomes the solution set, as one sees instantly from (2.1.2).

2.1.2 Pivoting

In “Gauss-Jordan elimination with no pivoting,” only the second operation in the above list is used. The zeroth row is divided by the element a_{00} (this being a trivial linear combination of the zeroth row with any other row — zero coefficient for the other row). Then the right amount of the zeroth row is subtracted from each other row to make all the remaining a_{i0} 's zero. The zeroth column of $\mathbf{A}$ now agrees with the identity matrix. We move to column 1 and divide row 1 by a_{11} , then subtract the right amount of row 1 from rows 0, 2, and 3, so as to make their entries in column 1 zero. Column 1 is now reduced to the identity form. And so on for columns 2 and 3. As we do these operations to $\mathbf{A}$, we of course also do the corresponding operations to the $\mathbf{b}$'s and to $\mathbf{1}$ (which by now no longer resembles the identity matrix in any way!).

Obviously we will run into trouble if we ever encounter a zero element on the (then current) diagonal when we are going to divide by the diagonal element. (The element that we divide by, incidentally, is called the *pivot element* or *pivot*.) Not so obvious, but true, is the fact that Gauss-Jordan elimination with no pivoting (no use of the first or third procedures in the above list) is numerically unstable in the presence of any roundoff error, even when a zero pivot is not encountered. You must *never* do Gauss-Jordan elimination (or Gaussian elimination; see below) without pivoting!

So what *is* this magic pivoting? Nothing more than interchanging rows (*partial pivoting*) or rows and columns (*full pivoting*), so as to put a particularly desirable element in the diagonal position from which the pivot is about to be selected. Since we don't want to mess up the part of the identity matrix that we have already built up, we can choose among elements that are both (i) on rows below (or on) the one that is about to be normalized, and also (ii) on columns to the right (or on) the column we are about to eliminate. Partial pivoting is easier than full pivoting, because we don't have to keep track of the permutation of the solution vector. Partial pivoting makes available as pivots only the elements already in the correct column. It turns out that partial pivoting is “almost” as good as full pivoting, in a sense that can be made mathematically precise, but which need not concern us here (for discussion and references, see [1]). To show you both variants, we do full pivoting in the routine in this section and partial pivoting in §2.3.

We have to state how to recognize a particularly desirable pivot when we see one. The answer to this is not completely known theoretically. It is known, both theoretically and in practice, that simply picking the largest (in magnitude) available element as the pivot is a very good choice. A curiosity of this procedure, however, is that the choice of pivot will depend on the original scaling of the equations. If we take the third linear equation in our original set and multiply it by a factor of a million, it is almost guaranteed that it will contribute the first pivot; yet the underlying solution

of the equations is not changed by this multiplication! One therefore sometimes sees routines which choose as pivot that element which *would* have been largest if the original equations had all been scaled to have their largest coefficient normalized to unity. This is called *implicit pivoting*. There is some extra bookkeeping to keep track of the scale factors by which the rows would have been multiplied. (The routines in §2.3 include implicit pivoting, but the routine in this section does not.)

Finally, let us consider the storage requirements of the method. With a little reflection you will see that at every stage of the algorithm, *either* an element of **A** is predictably a one or zero (if it is already in a part of the matrix that has been reduced to identity form) *or else* the exactly corresponding element of the matrix that started as **1** is predictably a one or zero (if its mate in **A** has not been reduced to the identity form). Therefore the matrix **1** does not have to exist as separate storage: The matrix inverse of **A** is gradually built up in **A** as the original **A** is destroyed. Likewise, the solution vectors **x** can gradually replace the right-hand side vectors **b** and share the same storage, since after each column in **A** is reduced, the corresponding row entry in the **b**'s is never again used.

Here is a routine that does Gauss-Jordan elimination with full pivoting, replacing its input matrices by the desired answers. Immediately following is an overloaded version for use when there are no right-hand sides, i.e., when you want only the matrix inverse.

gaussj.h

```
void gaussj(MatDoub_IO &a, MatDoub_IO &b)
```

Linear equation solution by Gauss-Jordan elimination, equation (2.1.1) above. The input matrix is $a[0..n-1][0..n-1]$. $b[0..n-1][0..m-1]$ is input containing the m right-hand side vectors. On output, a is replaced by its matrix inverse, and b is replaced by the corresponding set of solution vectors.

```
{
    Int i,icol,irow,j,k,l,ll,n=a.nrows(),m=b.ncols();
    Doub big,dum,pivinv;
    VecInt indxc(n),indxr(n),ipiv(n);
    for (j=0;j<n;j++) ipiv[j]=0;
    for (i=0;i<n;i++) {
        big=0.0;
        for (j=0;j<n;j++)
            for (k=0;k<n;k++)
                if (ipiv[j] != 1)
                    if (ipiv[k] == 0) {
                        if (abs(a[j][k]) >= big) {
                            big=abs(a[j][k]);
                            irow=j;
                            icol=k;
                        }
                    }
        ++(ipiv[icol]);
        We now have the pivot element, so we interchange rows, if needed, to put the pivot
        element on the diagonal. The columns are not physically interchanged, only relabeled:
        indxc[i], the column of the (i + 1)th pivot element, is the (i + 1)th column that is
        reduced, while indxr[i] is the row in which that pivot element was originally located.
        If indxr[i] ≠ indxc[i], there is an implied column interchange. With this form of
        bookkeeping, the solution b's will end up in the correct order, and the inverse matrix
        will be scrambled by columns.
        if (irow != icol) {
            for (l=0;l<n;l++) SWAP(a[irow][l],a[icol][l]);
            for (l=0;l<m;l++) SWAP(b[irow][l],b[icol][l]);
        }
        indxr[i]=irow;
```

We are now ready to divide the pivot row by the pivot element, located at $irow$ and $icol$.

```

indxc[i]=icol;
if (a[icol][icol] == 0.0) throw("gaussj: Singular Matrix");
pivinv=1.0/a[icol][icol];
a[icol][icol]=1.0;
for (l=0;l<n;l++) a[icol][l] *= pivinv;
for (l=0;l<m;l++) b[icol][l] *= pivinv;
for (ll=0;ll<n;ll++)
    if (ll != icol) {
        dum=a[ll][icol];
        a[ll][icol]=0.0;
        for (l=0;l<n;l++) a[ll][l] -= a[icol][l]*dum;
        for (l=0;l<m;l++) b[ll][l] -= b[icol][l]*dum;
    }
}
This is the end of the main loop over columns of the reduction. It only remains to unscramble
the solution in view of the column interchanges. We do this by interchanging pairs of
columns in the reverse order that the permutation was built up.
for (l=n-1;l>=0;l--) {
    if (indxr[l] != indxc[l])
        for (k=0;k<n;k++)
            SWAP(a[k][indxr[l]],a[k][indxc[l]]);
}
And we are done.
}

void gaussj(MatDoub_IO &a)
Overloaded version with no right-hand sides. Replaces a by its inverse.
{
    MatDoub b(a.nrows(),0);
    gaussj(a,b);
}
Dummy vector with zero columns.

```

2.1.3 Row versus Column Elimination Strategies

The above discussion can be amplified by a modest amount of formalism. Row operations on a matrix $\mathbf{A}$ correspond to pre- (that is, left-) multiplication by some simple matrix $\mathbf{R}$. For example, the matrix $\mathbf{R}$ with components

$$R_{ij} = \begin{cases} 1 & \text{if } i = j \text{ and } i \neq 2, 4 \\ 1 & \text{if } i = 2, j = 4 \\ 1 & \text{if } i = 4, j = 2 \\ 0 & \text{otherwise} \end{cases} \quad (2.1.5)$$

effects the interchange of rows 2 and 4. Gauss-Jordan elimination by row operations alone (including the possibility of *partial* pivoting) consists of a series of such left-multiplications, yielding successively

$$\begin{aligned} \mathbf{A} \cdot \mathbf{x} &= \mathbf{b} \\ (\cdots \mathbf{R}_2 \cdot \mathbf{R}_1 \cdot \mathbf{R}_0 \cdot \mathbf{A}) \cdot \mathbf{x} &= \cdots \mathbf{R}_2 \cdot \mathbf{R}_1 \cdot \mathbf{R}_0 \cdot \mathbf{b} \\ (\mathbf{I}) \cdot \mathbf{x} &= \cdots \mathbf{R}_2 \cdot \mathbf{R}_1 \cdot \mathbf{R}_0 \cdot \mathbf{b} \\ \mathbf{x} &= \cdots \mathbf{R}_2 \cdot \mathbf{R}_1 \cdot \mathbf{R}_0 \cdot \mathbf{b} \end{aligned} \quad (2.1.6)$$

The key point is that since the $\mathbf{R}$'s build from right to left, the right-hand side is simply transformed at each stage from one vector to another.

Column operations, on the other hand, correspond to post-, or right-, multiplications by simple matrices, call them $\mathbf{C}$. The matrix in equation (2.1.5), if right-multiplied onto a matrix $\mathbf{A}$, will interchange *columns* 2 and 4 of $\mathbf{A}$. Elimination by column operations involves (conceptually) inserting a column operator, *and also its inverse*, between the matrix $\mathbf{A}$ and the